# CEVE 543 Fall 2025 Lab 7: Bias Correction Implementation

## Delta method and quantile mapping for temperature bias correction

CEVE 543 Fall 2025

2025-10-24

# 1 Background and Goals

Climate models have systematic biases that directly affect impact assessments, arising from coarse spatial resolution, parameterization of sub-grid processes, representation of topography, and errors in simulated circulation patterns. This lab implements two widely-used bias correction approaches: the delta method and quantile-quantile (QQ) mapping. The delta method preserves the climate model's change signal while anchoring absolute values to observations, whereas QQ-mapping corrects the full distribution of values.

Both methods assume stationarity—that the statistical relationship between model and observations remains constant across climate states. This assumption may not hold under significant climate change. We'll explore the strengths and limitations of each method using temperature data for Boston, providing hands-on experience before PS2 Part 1.

# 2 Study Location and Data

This lab uses temperature data for Boston Logan International Airport (Station ID: USW00014739, 42.3631°N, 71.0064°W). Observational data comes from GHCN-Daily (1936-2024) and is provided in the `USW00014739.csv` file in degrees Celsius. Climate model data comes from the GFDL-ESM4 model's 3-hourly near-surface air temperature (`tas`), pre-downloaded from Google Cloud Storage for both historical (1850-2014) and SSP3-7.0 (2015-2100) scenarios. Refer to Lab 6 for details on CMIP6 data structure.

## 2.1 Data Processing Notes

The GHCN data provides daily average temperature in degrees Celsius. CMIP6 provides 3-hourly instantaneous temperature in Kelvin, which we'll convert to Celsius and aggregate to daily averages. The pre-downloaded NetCDF files (`boston_historical.nc` and `boston_ssp370.nc`) contain 3-hourly surface air temperature for the grid cell nearest to Boston. We aggregate 8 consecutive 3-hour periods to approximate daily averages, ignoring daylight saving time—a simplification typical in many bias correction applications. If you're interested in applying these methods to a different location, the `download_data.jl` script demonstrates how to extract CMIP6 data from Google Cloud Storage.

! Before Starting

Before starting the lab, uncomment the `Pkg.instantiate()` line in the first code block and run it to install all required packages. This will take a few minutes the first time. After installation completes, comment the line back out to avoid reinstalling on subsequent runs.

# 3 Lab Implementation

## 3.1 Package Setup

```
using Pkg
lab_dir = dirname(@__FILE__)
Pkg.activate(lab_dir)
Pkg.instantiate() # uncomment this the first time you run the lab to install
packages, then comment it back
```

```
using CSV, CairoMakie, DataFrames, Dates, LaTeXStrings, NCDatasets, Statistics,
StatsBase, TidierData
ENV["DATAFRAMES_ROWS"] = 6
CairoMakie.activate!()

# Constants for plotting
const MONTH_NAMES = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep",
"Oct", "Nov", "Dec"]
const MONTH_LABELS = (1:12, MONTH_NAMES)
```

## 3.2 Task 1: Load and Process Data

We begin by loading both observational and climate model data. The observational data from GHCN-Daily spans 1936-2024, while the CMIP6 data requires processing from 3-hourly to daily resolution. We'll use the overlapping historical period (1995-2014) to calibrate bias corrections.

! Instructions

Load the Boston GHCN temperature data from `USW00014739.csv` and filter to years with at least 80% complete data. Then load the pre-downloaded CMIP6 NetCDF files and aggregate the 3-hourly temperature data to daily values for the historical (1995-2014) and near-term future (2020-2040) periods. The helper functions `load_cmip6_data()` and `aggregate_to_daily()` are provided below. Finally, visualize the annual cycle comparing observations vs the historical GCM simulation.

```
# Load and clean the observational data
data_path = joinpath(lab_dir, "USW00014739.csv")
df = @chain begin
    CSV.read(data_path, DataFrame)
    @mutate(
        TAVG = ifelse.(ismissing.(TMIN) .| ismissing.(TMAX), missing, (TMIN .+
```

```
TMAX) ./ 2),
    )
    @mutate(TAVG = TAVG / 10.0) # Convert to degrees C
    @rename(date = DATE)
    @mutate(year = year(date), month = month(date))
    @select(date, year, month, TAVG)
end

# Filter to years with at least 80% complete data
yearly_counts = @chain df begin
    @group_by(year)
    @summarize(n_obs = sum(!ismissing(TAVG)), n_total = n())
    @mutate(frac_complete = n_obs / n_total)
end

good_years_df = @chain yearly_counts begin
    @filter(frac_complete >= 0.8)
end
good_years = good_years_df.year

df_clean = @chain df begin
    @filter(year in !!good_years)
    dropmissing(:TAVG)
end
```

```
# Helper functions for CMIP6 data
"""
Load CMIP6 temperature data from a local NetCDF file and convert times to DateTime.
"""
function load_cmip6_data(file_path::String)
    ds = NCDataset(file_path)
    tas_data = ds["tas"][:]
    time_cf = ds["time"][:]
    close(ds)

    time_data = [DateTime(
        Dates.year(t), Dates.month(t), Dates.day(t),
        Dates.hour(t), Dates.minute(t), Dates.second(t)
    ) for t in time_cf]

    return tas_data, time_data
end

"""
Aggregate 3-hourly temperature data to daily averages.
"""
function aggregate_to_daily(tas_3hr, time_3hr)
    n_3hr_per_day = 8
    n_days = div(length(tas_3hr), n_3hr_per_day)
```

```
    daily_temp = Vector{Float64}(undef, n_days)
    daily_dates = Vector{Date}(undef, n_days)

    for i in 1:n_days
        idx_start = (i - 1) * n_3hr_per_day + 1
        idx_end = i * n_3hr_per_day
        daily_vals = collect(skipmissing(tas_3hr[idx_start:idx_end]))
        daily_temp[i] = isempty(daily_vals) ? NaN : mean(daily_vals)
        daily_dates[i] = Date(time_3hr[idx_start])
    end

    daily_temp_c = daily_temp .- 273.15  # Convert K to C
    return daily_temp_c, daily_dates
end

# Load and process CMIP6 data
hist_file = joinpath(lab_dir, "boston_historical.nc")
ssp370_file = joinpath(lab_dir, "boston_ssp370.nc")

tas_hist_3hr, time_hist_3hr = load_cmip6_data(hist_file)
tas_ssp370_3hr, time_ssp370_3hr = load_cmip6_data(ssp370_file)

# Process historical period (1995-2014)
hist_start = DateTime(1995, 1, 1)
hist_end = DateTime(2014, 12, 31, 23, 59, 59)
hist_idx = (time_hist_3hr .>= hist_start) .& (time_hist_3hr .<= hist_end)
tas_hist_daily, dates_hist_daily = aggregate_to_daily(tas_hist_3hr[hist_idx],
time_hist_3hr[hist_idx])

# Process near-term period (2020-2040)
near_start = DateTime(2020, 1, 1)
near_end = DateTime(2040, 12, 31, 23, 59, 59)
near_idx = (time_ssp370_3hr .>= near_start) .& (time_ssp370_3hr .<= near_end)
tas_ssp370_near_daily, dates_ssp370_near_daily =
aggregate_to_daily(tas_ssp370_3hr[near_idx], time_ssp370_3hr[near_idx])

# Create DataFrames
df_gcm_hist = DataFrame(
    date=dates_hist_daily, temp=tas_hist_daily,
    year=year.(dates_hist_daily), month=month.(dates_hist_daily)
)

df_ssp370_near = DataFrame(
    date=dates_ssp370_near_daily, temp=tas_ssp370_near_daily,
    year=year.(dates_ssp370_near_daily), month=month.(dates_ssp370_near_daily)
)

df_obs_hist = @chain df_clean begin
    @filter(year >= 1995 && year <= 2014)
end
```

```
# Compute monthly climatologies
obs_monthly = @chain df_obs_hist begin
    @group_by(month)
    @summarize(mean_temp = mean(TAVG))
end

gcm_monthly = @chain df_gcm_hist begin
    @group_by(month)
    @summarize(mean_temp = mean(temp))
end
```

```
let
    fig = Figure()
    ax = Axis(fig[1, 1],
        xlabel="Month",
        ylabel=L"Temperature ($^\circ$C)",
        title="Annual Cycle: Observations vs GCM Historical (1995-2014)",
        xticks=MONTH_LABELS)
    lines!(ax, obs_monthly.month, obs_monthly.mean_temp, linewidth=2,
color=:steelblue, label="Observations")
    lines!(ax, gcm_monthly.month, gcm_monthly.mean_temp, linewidth=2, color=:coral,
label="GCM Historical")
    axislegend(ax, position=:lt)
    fig
end
```

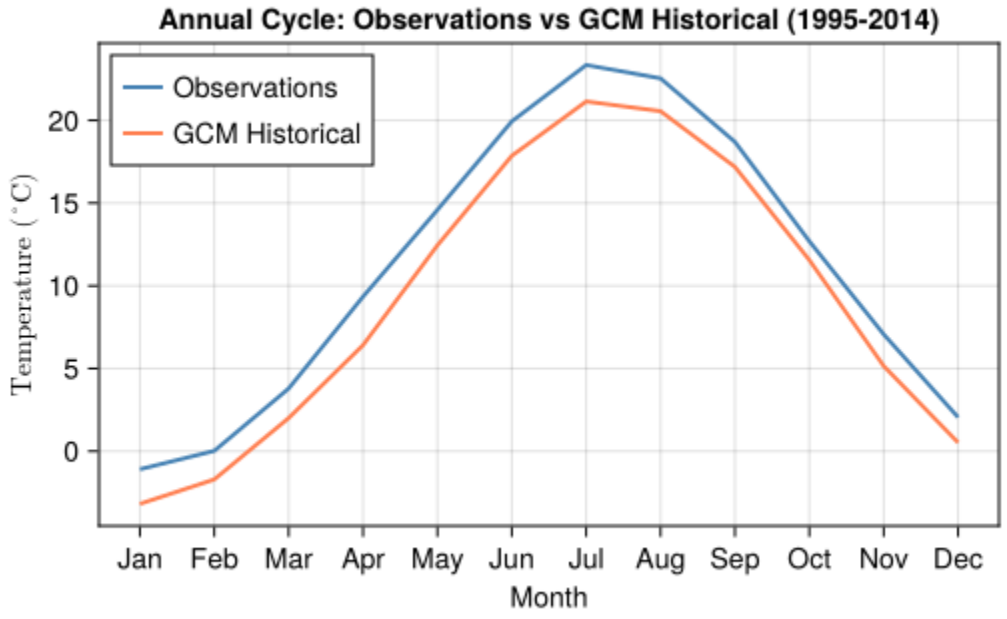



Figure 1: Annual cycle comparison between GHCN observations and GFDL-ESM4 historical simulation for Boston (1995-2014). The GCM shows a warm bias across most months.

### 3.3 Task 2: Implement Delta Method

The delta method corrects the mean bias while preserving the climate model's projected change signal. We calculate a monthly bias correction based on the historical period (1995-2014) and apply it to future projections.

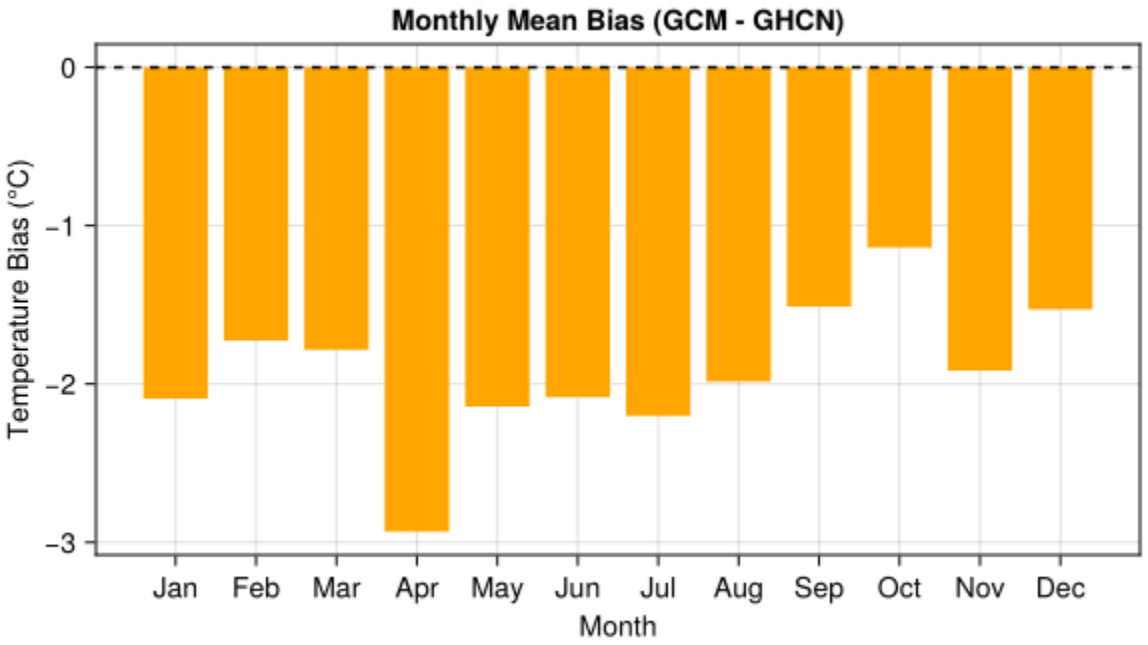
> ! Instructions
>
> Implement the additive delta method for temperature bias correction. For each calendar month $m$, calculate the mean bias: $\Delta_m = \bar{T}^{\text{GCM}}_{\text{hist},m} - \bar{T}^{\text{obs}}_{\text{hist},m}$. Then apply the correction to future values: $T^{\text{corr}}_{\text{fut}}(d, m, y) = T^{\text{GCM}}_{\text{fut}}(d, m, y) - \Delta_m$.
>
> Follow these steps:
>
> 1. Calculate the monthly mean bias by grouping both `df_gcm_hist` and `df_obs_hist` by month, computing their means, joining them, and computing the difference.
> 2. Create a bar plot visualizing the monthly bias.
> 3. Write a function `apply_delta_method(gcm_temps, gcm_dates, monthly_bias_df)` that applies the bias correction to a vector of temperatures and dates.
> 4. Apply your function to the near-term data and add a new column `temp_delta` to `df_ssp370_near`.
> 5. Create monthly climatologies for both raw and delta-corrected near-term data.
> 6. Visualize the annual cycle showing historical observations, raw GCM near-term, and delta-corrected near-term temperatures.

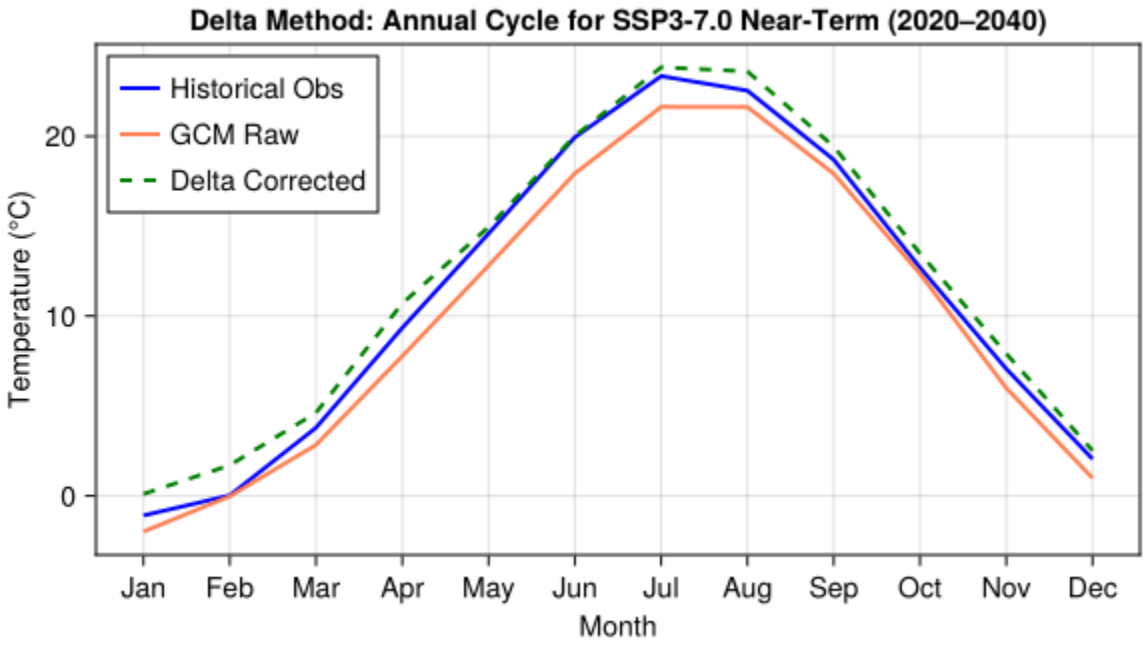
```
# Delta method: Here we will find the diff of observations(GHCN) and hitorical
(CMIP6) for each month, then we will add that to SSP 2020 to 2040, each month....say
for July the GCM hist(CMIP6) is saying the temp is 21 degree, the GHCN obs is saying
23 degree, that means the diff is -2 degree...so now we will add 2 degree to each
year july value of SSP...say 2030 SSP july is saying 31 degree, so after adding 2, it
will be 33 degree..(note if the diff was opposite, say +2 degree, then we will
substract 2 from 31 that means 31-2=29 degree)
# ------------------------------------------------------------------
# 3.3 Task 2: Implement Additive Delta Method
# ------------------------------------------------------------------
using CairoMakie
CairoMakie.activate!()
using CairoMakie, DataFrames, Dates, Statistics

# Calculate monthly mean bias (Δm = GCM_hist - OBS_hist)
monthly_bias = DataFrame(
    month = gcm_monthly.month,
    gcm_mean = gcm_monthly.mean_temp,
    obs_mean = obs_monthly.mean_temp
)
monthly_bias.bias = monthly_bias.gcm_mean .- monthly_bias.obs_mean

# Print first few rows to verify
println("Monthly Bias Table (Δm = GCM - OBS):")
show(first(monthly_bias, 12), allrows=true, allcols=true)
```

```
# Visualize the bias using CairoMakie barplot
fig_bias = Figure(size = (600, 350))
ax_bias = Axis(fig_bias[1, 1],
    title = "Monthly Mean Bias (GCM - GHCN)",
    xlabel = "Month",
    ylabel = "Temperature Bias (°C)",
    xticks = MONTH_LABELS
)
barplot!(ax_bias, monthly_bias.month, monthly_bias.bias, color = :orange)
hlines!(ax_bias, [0], color = :black, linestyle = :dash)
display(fig_bias)

# Write the correction function
function apply_delta_method(gcm_temps::Vector{Float64}, gcm_dates::Vector{Date},
monthly_bias_df::DataFrame)
    corrected_temps = similar(gcm_temps)
    for i in eachindex(gcm_temps)
        m = month(gcm_dates[i])
        Δm = monthly_bias_df.bias[monthly_bias_df.month .== m][1]
        corrected_temps[i] = gcm_temps[i] - Δm
    end
    return corrected_temps
end

# Apply the correction to SSP3-7.0 (2020–2040)
df_ssp370_near.temp_delta = apply_delta_method(df_ssp370_near.temp,
                                               df_ssp370_near.date,
                                               monthly_bias)

# Compute monthly climatologies (no macros)
near_monthly_raw = combine(groupby(df_ssp370_near, :month),
                           :temp => mean => :mean_temp)
near_monthly_delta = combine(groupby(df_ssp370_near, :month),
                             :temp_delta => mean => :mean_temp)

# Create comparison plot (CairoMakie)
fig_delta = Figure(resolution = (600, 350))
ax_delta = Axis(fig_delta[1, 1],
    title = "Delta Method: Annual Cycle for SSP3-7.0 Near-Term (2020–2040)",
    xlabel = "Month",
    ylabel = "Temperature (°C)",
    xticks = MONTH_LABELS
)

lines!(ax_delta, obs_monthly.month, obs_monthly.mean_temp,
    color = :blue, linewidth = 2, label = "Historical Obs")
lines!(ax_delta, near_monthly_raw.month, near_monthly_raw.mean_temp,
    color = :coral, linewidth = 2, label = "GCM Raw")
lines!(ax_delta, near_monthly_delta.month, near_monthly_delta.mean_temp,
    color = :green, linewidth = 2, linestyle = :dash, label = "Delta Corrected")
```

```
axislegend(ax_delta, position = :lt)
fig_bias
fig_delta
```

```
Monthly Bias Table (Δm = GCM - OBS):
12×4 DataFrame
 Row │ month  gcm_mean  obs_mean    bias
     │ Int64  Float64   Float64     Float64
─────┼──────────────────────────────────────
   1 │     1  -3.18274  -1.09024    -2.0925
   2 │     2  -1.70476   0.0218584  -1.72662
   3 │     3   2.00539   3.78944    -1.78405
   4 │     4   6.41217   9.34408    -2.93191
   5 │     5  12.445    14.5874     -2.14243
   6 │     6  17.8571   19.9404     -2.08332
   7 │     7  21.1401   23.3412     -2.20107
   8 │     8  20.5512   22.5353     -1.9841
   9 │     9  17.1836   18.6946     -1.51102
  10 │    10  11.555    12.6931     -1.1381
  11 │    11   5.14819   7.06367    -1.91548
  12 │    12   0.52005   2.04863    -1.52858┌ Warning: Found `resolution` in the
theme when creating a `Scene`. The `resolution` keyword for `Scene`s and `Figure`s
has been deprecated. Use `Figure(; size = ...` or `Scene(; size = ...)` instead,
which better reflects that this is a unitless size and not a pixel resolution. The
key could also come from `set_theme!` calls or related theming functions.
└ @ Makie C:\Users\Owner\.julia\packages\Makie\4JW9B\src\scenes.jl:264
```






### 3.4 Task 3: Implement Quantile-Quantile Mapping

Unlike the delta method, QQ-mapping transforms the entire probability distribution of model output to match observations. For each value in the future model output, we find its percentile in the historical model distribution, then map it to the same percentile in the historical observed distribution.

**! Instructions**

Implement QQ-mapping to correct the full distribution of temperature values.

Follow these steps:

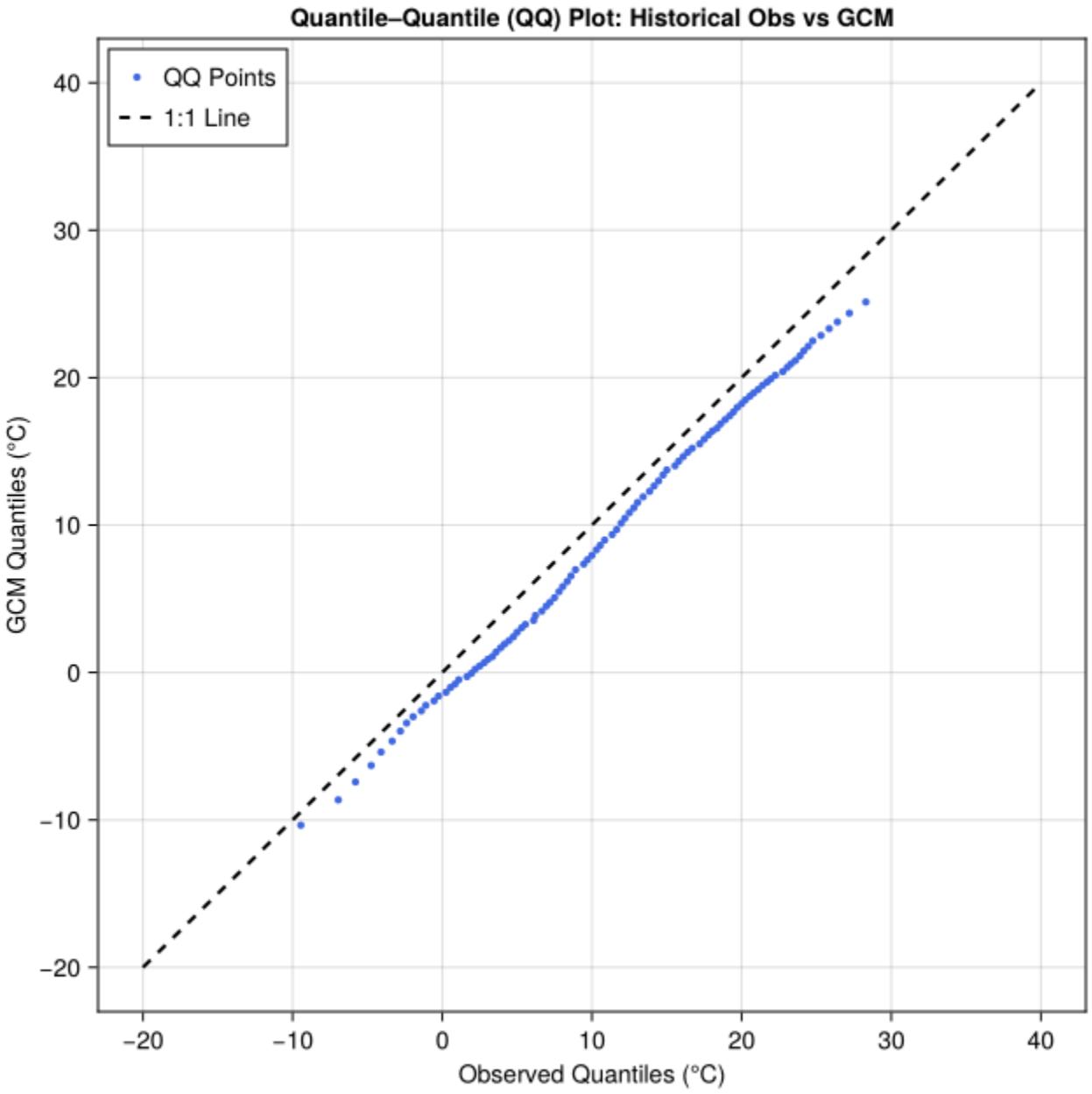
1. Extract the observed and GCM historical temperatures as vectors (`obs_hist_temps` and `gcm_hist_temps`).
2. Use `ecdf()` from StatsBase.jl to fit empirical cumulative distribution functions to both datasets.
3. Write a function `apply_qqmap_empirical(gcm_temps, ecdf_gcm, obs_hist_temps)` that:
   - For each temperature value, finds its percentile in the GCM CDF
   - Clamps the percentile to the range [0.001, 0.999] to avoid extrapolation
   - Maps to the same percentile in the observed distribution using `quantile()`
4. Apply your function to the near-term data and add a new column `temp_qqmap` to `df_ssp370_near`.
5. Create a QQ-plot comparing observed and GCM quantiles for the historical period, showing the 1:1 line.

**Hints**

- Use `ecdf_gcm(value)` to get the percentile of a value in the GCM distribution
- Use `quantile(obs_hist_temps, p)` to get the value at percentile `p` in the observed distribution
- The `clamp(x, low, high)` function constrains `x` to the range [low, high]

```
# quantile-quantile plot
###say the Observed (Reality) 0, 5, 10, 15, 20
###GCM (Model) -2, 3, 8, 13, 18
###GCM 50th percentile is 8, the model 50th percentile is 10, so ###replace the GCM 8
with the value 10
using DataFrames, StatsBase, CairoMakie, Statistics

# --- 1. Extract temperature vectors ---
# Convert columns to Float64 vectors (ensuring no missing values)
obs_hist_temps = Float64.(skipmissing(df_obs_hist.TAVG))
gcm_hist_temps = Float64.(skipmissing(df_gcm_hist.temp))

# --- 2. Fit empirical CDFs (ECDF) ---
ecdf_obs = ecdf(obs_hist_temps)     # Observed distribution
ecdf_gcm = ecdf(gcm_hist_temps)     # Historical GCM distribution

# --- 3. Define the QQ-mapping function ---
function apply_qqmap_empirical(gcm_temps::Vector{Float64}, ecdf_gcm,
obs_hist_temps::Vector{Float64})
    corrected_temps = similar(gcm_temps)
    for i in eachindex(gcm_temps)
        # Find percentile of each GCM temperature within its own historical CDF
        p = ecdf_gcm(gcm_temps[i])
        # Clamp the percentile to avoid edge issues (0 or 1)
```

```
        p = clamp(p, 0.001, 0.999)
        # Map to corresponding temperature in the observed distribution
        corrected_temps[i] = quantile(obs_hist_temps, p)
    end
    return corrected_temps
end

# --- 4. Apply the correction to future (SSP3–7.0) GCM data ---
df_ssp370_near.temp_qqmap = apply_qqmap_empirical(
    Float64.(skipmissing(df_ssp370_near.temp)),
    ecdf_gcm,
    obs_hist_temps
)

# --- 5. Create QQ-plot (Observed vs Historical GCM) ---
percentiles = 0.01:0.01:0.99
obs_quantiles = quantile(obs_hist_temps, percentiles)
gcm_hist_quantiles = quantile(gcm_hist_temps, percentiles)

# Activate CairoMakie backend
CairoMakie.activate!()

# Create the QQ plot
fig = Figure(size = (650, 650))
ax = Axis(fig[1, 1],
    title = "Quantile–Quantile (QQ) Plot: Historical Obs vs GCM",
    xlabel = "Observed Quantiles (°C)",
    ylabel = "GCM Quantiles (°C)"
)

# Scatter the quantiles (distribution comparison)
CairoMakie.scatter!(ax, obs_quantiles, gcm_hist_quantiles,
    color = :royalblue, markersize = 6, label = "QQ Points")

# Add 1:1 reference line (perfect distribution match)
CairoMakie.lines!(ax, [-20, 40], [-20, 40],
    color = :black, linestyle = :dash, linewidth = 2, label = "1:1 Line")

axislegend(ax, position = :lt)
display(fig)
```




```
CairoMakie.Screen{IMAGE}
```

### 3.5 Task 4: Compare Methods

Now we compare the delta method and QQ-mapping approaches to understand their strengths and limitations.

! Instructions

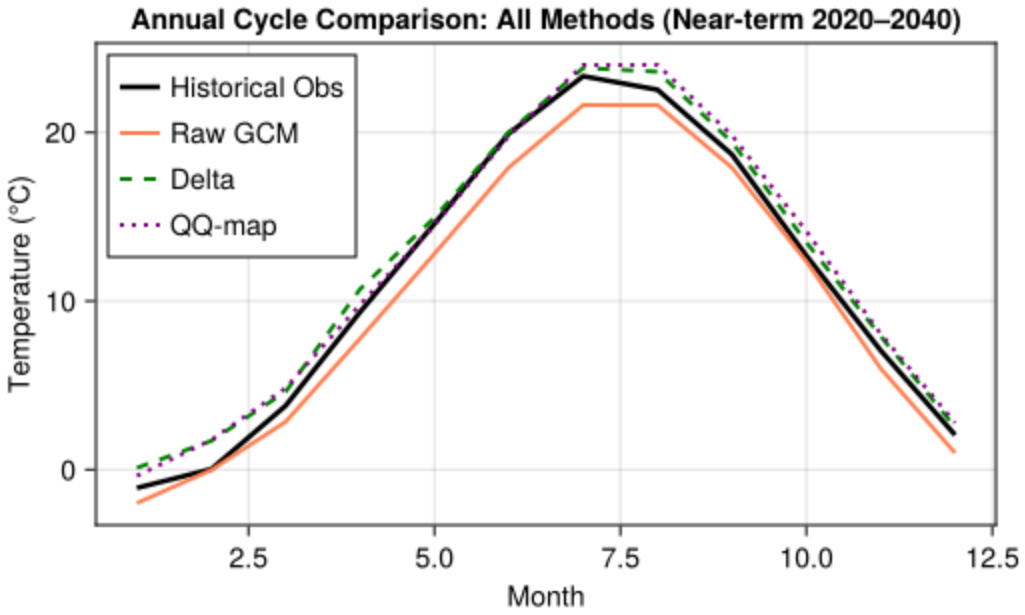
Create a single figure showing monthly mean temperature for the near-term period (2020-2040) using all four approaches:

1. Compute the monthly mean for QQ-mapped temperatures (`near_monthly_qqmap`)
2. Create a figure with 4 lines:
   - Historical observations (`obs_monthly`)
   - Raw GCM near-term (`near_monthly_raw`)
   - Delta-corrected near-term (`near_monthly_delta`)
   - QQ-mapped near-term (`near_monthly_qqmap`)

After creating the plot, examine it carefully and consider:

- How do the two correction methods differ? Ans: During the start of the x axis, the delta green line stays a bit higher than the QQ mapping, which says the delta-corrected temperatures are a bit higher(strong warming) than the QQ plot during the months of April-June.
- What does the QQ-mapped line reveal about this method's treatment of the warming signal? Ans: The QQ line shows less warming than the delta method during months April-June. QQ line fixes the values based on where it sits in the distribution, but not just by a fix value like delta method.

```
# Your code here
using DataFrames, Statistics
###compute monthly mean
near_monthly_qqmap = combine(groupby(df_ssp370_near, :month),
    :temp_qqmap => mean => :mean_temp)

###creating figures
using CairoMakie

fig, ax = lines(
    obs_monthly.month, obs_monthly.mean_temp,
    color = :black, linewidth = 2.5, label = "Historical Obs"
)

lines!(ax, near_monthly_raw.month, near_monthly_raw.mean_temp,
    color = :coral, linewidth = 2, label = "Raw GCM")

lines!(ax, near_monthly_delta.month, near_monthly_delta.mean_temp,
    color = :green, linestyle = :dash, linewidth = 2, label = "Delta")

lines!(ax, near_monthly_qqmap.month, near_monthly_qqmap.mean_temp,
    color = :purple, linestyle = :dot, linewidth = 2, label = "QQ-map")

ax.xlabel = "Month"
ax.ylabel = "Temperature (°C)"
ax.title = "Annual Cycle Comparison: All Methods (Near-term 2020–2040)"
axislegend(ax, position = :lt)
display(fig)
```




```
CairoMakie.Screen{IMAGE}
```

## 3.6 Task 5: Reflection

Finally, we reflect on the assumptions, limitations, and appropriate use cases for each bias correction method.

! Instructions

Write brief responses (2-3 sentences each) to the following questions:

1. **Method selection:** If you were providing climate data to support a decision about urban heat management in Boston, which bias correction method would you recommend and why?
2. **Appropriateness of QQ-mapping:** In the lines_biascorrection:2006 paper we discussed, QQ-mapping was used to correct both the *frequency* and *intensity* of rainfall for crop modeling. For temperature data, does it make sense to correct the full distribution? Why or why not? When might the delta method be more appropriate than QQ-mapping?

### 3.6.a Method Selection for Urban Heat Management

Ans: For urban heat management, I would prefer the delta method, because urban heat managment primarily focus on mean temperature trend and delta method is sufficient for that. Also, this delta method is much easier to implement for decision makers. So, my takeway is mainly, Q-Q plot is for rainfall and delta method is for heat management.

#### 3.6.b Appropriateness of QQ-Mapping for Temperature

Ans: The temp data, if we want to know the long term warming trend, then the mean correction or delta method is suitable. The delta method only adjusts the mean by shifting entire dataset up or down while it keeps the future warming signal of the climate model unchanged.

Menawhile, QQ map change the full shape of the temp distribution. So, if we care about the short term temp variability, then the QQ method is suitable so thatwe can get more accurate daily or extreme temp ptterns.

# 4 References